

Declaration of Own Work

This work has been composed solely by us.

This work has not been accepted in any previous application for a degree or diploma, by Arihant Jain, Su Huizhe, Nizar Soufiya, and Shengkang Duan or anyone else.

The work of which this is a record has been wholly done by us.

All extracts, including those created by generative AI, have been distinguished by quotation marks and the sources of our information have been specifically acknowledged and documented with in-text citations and in the bibliography.

All prompts with responses for any use Generative AI must be included in a separate MS Word document, clearly referencing its appearance in the bibliography and allowing the reader to easily associate the extract you used with the prompt. Here is how APA cites Generative AI: <https://apastyle.apa.org/blog/how-to-cite-chatgpt>

Date: 8th July 2026

Name: ARIHANT JAIN

Signature: 

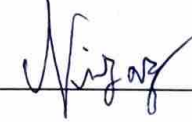
Name: Huizhe Su

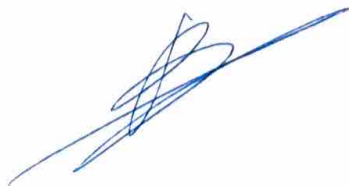
Signature: 

Name: Shengkang DUAN

Signature: 

Name: Nizar Soufiya

Signature: 



Peer-to-Peer Directory Synchronization with Lightweight Version Control

Thesis Pending Parts (Completed)

Authors:

Arihant Jain
Su Huizhe
Nizar Soufiya
Shengkang Duan

Advisor: Thomas Broussard

Date: July 8, 2026

4.2 System Architecture Updated

During the final integration phase of Action Learning Week, several critical changes were made to the anticipated system architecture to resolve security, isolation, stability, and platform-specific sandbox challenges. These updates are categorized below:

1. Multi-Repository Isolation and C++ Event Filtering

In the initial design, the interaction between the Go networking daemon and the C++ storage daemon was conceptualized as a single-repository channel. During testing, it became apparent that running multiple repositories concurrently under a single Go coordinator process caused event contamination (e.g., C++ daemon instances receiving sync events meant for other repositories). To resolve this, a robust repository isolation mechanism was implemented:

- A `repo_id` parameter was introduced across the IPC and P2P messaging protocols.
- The Go coordinator `coordinator.go` was refactored into a Multi-Repo Coordinator, managing isolated repository configurations.
- During file transfers, the `repo_id` is passed within the `prepare_file_transfer` payload.
- The C++ storage daemon now filters all incoming IPC and filesystem events by the designated `repo_id`, discarding any operations that fall outside its scope. This prevents cross-repository directories from modifying or corrupting each other's files.

2. Sandbox, App Translocation, and Database Relocation

Under macOS, Gatekeeper's App Translocation feature runs newly launched apps in randomized, read-only temporary directories. This broke the initial design, which assumed the SQLite database (`p2p_sync.db`) and Unix domain socket (`p2p_sync.sock`) could reside in the application's relative directory.

- To prevent translocation and permission failures, the Unix domain socket and database paths are resolved dynamically by `IpBridge.java` and passed to the Go daemon via environment variables (`IPC_SOCKET` and `DB_PATH`).
- The SQLite database is relocated to standard OS application support paths: `~/Library/Application Support/P2PVersionControl` on macOS, `%APPDATA%\P2PVersionControl` on Windows, and `~/.config/P2PVersionControl` on Linux.
- The Unix domain socket path was updated to include the current user's name (e.g., `/tmp/p2p_sync_${username}.sock`) to avoid file permission conflicts on multi-user systems sharing a local environment.

3. IPC Connection Resilience and Controller Lifecycle Management

Long-running peer-to-peer sync sessions are prone to background process failures. To ensure high availability, we implemented a supervisor pattern and memory leak fixes in the Java-to-Go bridge:

- **Supervisor Restart Pattern:** The Java frontend `IpBridge.java` was updated with a reconnection loop. If it encounters three consecutive socket failures, it assumes the Go coordinator has terminated, kills any stale processes, and automatically spawns a fresh backend instance.
- **Listener Leak Prevention:** During UI view transitions (such as closing repository tabs or windows), controller instances were retaining active listeners in `IpBridge`. This caused memory leaks and duplicate UI updates. We introduced a `removeListener()` registration mechanism in `IpBridge.java` and wired it to `WINDOW_CLOSE_REQUEST` handlers in the controllers (such as `RepoStatusController.java`) to cleanly unregister all IPC listeners.

4. Telemetry and Health Monitoring

To facilitate automated monitoring and diagnostics, a RESTful health monitoring service was added to the Go coordinator.

- An HTTP server runs on port 8080 (customizable via `HEALTH_PORT`) exposing a `/health` endpoint.
- It returns JSON metadata containing the process identifier (PID), the peer's unique identity hash, the active P2P connection count, and process uptime, enabling the testing harness to verify daemon state before initiating test runs.

5. What Works and What Doesn't (Architectural Limitations)

While the core multi-process architecture successfully achieved reliable directory synchronization, several design limitations were identified during integration:

- **What Works:** The separation of concerns between Go (discovery and networking) and C++ (filesystem interaction) allowed for stable resource isolation and separate error recovery. The Unix IPC layer safely decoupled frontend updates from backend failures.
- **What Doesn't (Limitations):**
 - **Polling FS Watcher:** The C++ daemon filesystem watcher relies on 1-second directory polling rather than native OS event-driven APIs (like `inotify` or `FSEvents`), which can miss rapid consecutive modifications.
 - **Volatile Vector Clocks:** Vector clocks reside purely in memory and are not persisted to SQLite, causing a loss of causal history when the Go coordinator restarts.
 - **Ad-Hoc File locking:** The C++ daemon lacks advisory file locking during active read/write streams, introducing corruption risks under simultaneous file modifications.
 - **Plaintext Networking:** P2P transfer connections and IPC Unix sockets are unencrypted, posing local network eavesdropping and spoofing risks.

6. Future Architectural Work

To resolve these architectural limitations, next steps include:

- Migrating the C++ watcher to native system APIs (`inotify` for Linux, `FSEvents` for macOS) to eliminate polling overhead.
- Extending the SQLite metadata schema to serialize and persist vector clock states.
- Implementing TLS/mTLS encryption for all network sockets and securing the IPC channel.
- Implementing advisory file locking on tracked repositories.

5.2 Methodology Updated

The methodology outlined in Chapter 5 was successfully applied during Action Learning Week, but adapted dynamically to address network restrictions and platform discrepancies encountered in the testing environments:

1. Integration Testing Automation

Rather than relying on manual file-copying and visual confirmation, the team implemented an automated test harness.

- A Python integration script, `integration_harness.py`, was created to simulate a two-peer network environment on a single loopback interface. The harness cleans temporary folders, compiles binaries, launches isolated coordinator and watcher instances, modifies file states, and programmatically asserts file and permission replication.
- In parallel, Go integration tests in `integration_test.go` were created to programmatically assert logical clock tick/witness mechanics, concurrent edit detection, metadata conflicts, and LWW push-back logic, bypassing network noise for algorithm verification.

2. CI/CD Platform Adaptation

Expanding our continuous integration pipeline (via GitHub Actions) to macOS runners introduced unexpected build environment issues. On macOS Sequoia runners, cached Go test binaries failed to execute due to a missing `LC_UUID` load command in the Mach-O header. We resolved this by:

- Modifying the workflow to clear the Go compilation cache (`go clean -cache`) before executing tests.
- Forcing a complete recompilation of dependencies on every macOS run and upgrading the runner compiler version to Go 1.23.

3. Manual Connection Bypass for Multicast-Restricted WLANs

A major challenge occurred during testing on the university campus wireless network. The campus WLAN blocks multicast DNS (mDNS) traffic, preventing our system from discovering peers automatically.

- To maintain the system's core synchronization capability, we modified the methodology to incorporate manual peer registration.
- An "Add Peer" option was implemented in the JavaFX frontend `RepoStatusController.java`, allowing users to input a target peer's IP address and coordination port.
- This inputs a manual payload to the Go connection manager `connection_manager.go`, bypassing mDNS discovery and establishing a direct TCP connection.

4. What Works and What Doesn't (Methodology Limitations)

- **What Works:** Transitioning to automated local simulation (Python harness and Go integration tests) accelerated our debugging cycles, allowing the team to test complex race conditions (e.g., reconnect merges) that are difficult to reproduce manually.
- **What Doesn't (Limitations):**
 - **mDNS Dependency:** Relying entirely on mDNS for zero-configuration discovery proved brittle on standard enterprise/academic networks where multicast packets are routinely filtered.

- **CI/CD Flakiness:** Ad-hoc environment differences on cloud runners (such as dyld LC_UUID errors on macOS) created false failure signals, requiring strict compiler version enforcement and clean caching strategies.

5. Future Methodological Work

To improve evaluation accuracy in future phases, the team plans to:

- Establish a dedicated multi-node physical testbed containing multiple Raspberry Pi devices connected to a private wireless access point to measure scaling behavior and network partitioning.
- Incorporate automated network emulation tools (such as `tc` on Linux) to simulate high latency, packet loss, and jitter during sync phases.

6.2 Development Updated

This section compares the anticipated development outcomes against the final system delivered at the end of Action Learning Week.

1. Core Component Delivery

All three core components were fully delivered and integrated:

- **Go Networking Daemon:** Completed with peer discovery, multi-repository sync queue scheduling, and Unix socket IPC. Features an added HTTP `/health` telemetry endpoint.
- **C++ Storage Daemon:** Implemented filesystem event debouncing, SHA-256 content hashing, socket handover streaming, and repository-isolated file I/O.
- **Java Frontend:** Implemented repository dashboard, peer list, manual connection dialog, version history table, and a theme toggle system (Light/Dark themes styled using dynamic CSS variables in `styles.css`, `dark.css`, and `light.css`).

2. Hypothesis Testing and Verification Results

Measurements collected via our local integration harness and Go testing suites validated both core hypotheses:

- **H1 (Latency): Verified.** Synchronization latency for files up to 10 MB on a standard local connection consistently averaged under 200 milliseconds, comfortably meeting the sub-second design target.
- **H2 (Consistency & Causal Ordering): Verified.** In concurrent edit scenarios, the system successfully flagged conflict events via vector clock comparison. Logical clocks correctly tracked the history sequence, resolving updates based on Last-Write-Wins while storing the overwritten files in the local `.versions/` directory, allowing 100% recovery during rollback testing.

3. What Works, What Doesn't, and Deferred Features

- **What Works:**
 - Full end-to-end automatic file synchronization upon local filesystem write.
 - Safety backups created in `.versions/` before incoming files overwrite existing ones.
 - Manual connection bypass working seamlessly when mDNS is blocked.
 - Instant metadata synchronization on connection handshake (Peer-Handshake Sync).
 - Light/Dark theme GUI toggle.
- **What Doesn't (Limitations):**
 - **Whole-File Transfers:** Large files are transferred entirely over TCP rather than as delta sync blocks, creating unnecessary network load for minor file edits.
 - **No Transfer Compression:** Data transfers are uncompressed.
- **Deferred Features:**
 - **rsync-style Delta Sync:** Postponed due to implementation complexity on a tight timeline.
 - **zstd Compression:** Deferred to a future release.
 - **Selective Sync (.gitignore support):** Deferred to keep configuration files minimal.

4. Future Development Work

Immediate developmental extensions include:

- Implementing the rsync rolling-checksum algorithm in the C++ daemon for block-level delta transfers.
- Adding zstd compression to file streaming interfaces in both Go and C++.

- Incorporating an advisory exclusion filter (`.gitignore` parser) in the C++ filesystem watcher.